

PROJECT 3 REPORT

Professor: Dr. Yicheng Tu

Parallel Bloom Filter with CUDA

Author: Divyansh Maurya

1. Introduction

A Bloom filter is a probabilistic data structure used to test set membership with no false negatives and a controllable false positive rate. It is compact, fast, and well-suited for parallelization because each element can be hashed independently.

This project implements a parallel Bloom filter on the GPU using CUDA. SipHash-2-4, a secure and fast 64-bit keyed hash function, is used for all hash computations.

A CPU implementation is also provided for baseline comparison. The objective is to evaluate the speedup achieved by GPU parallelization while maintaining correctness and zero false negatives.

2. Methodology

2.1 Data Generation

Random alphanumeric strings of varying length (1–20 characters) are generated at runtime. They are stored in a flattened 1D character buffer along with a position array indicating each string's starting offset.

This design matches the professor's CPU implementation and avoids excessive memory fragmentation.

2.2 CPU Bloom Filter

The CPU implementation uses the professor's provided SipHash reference code.

For each string:

1. SipHash is applied to compute multiple hashes.
2. Indices in the bit array are set to 1.
3. A second pass checks membership, recording any false negatives.

This establishes the baseline runtime for comparison.

2.3 GPU Bloom Filter

The GPU implementation assigns one thread per string.

Two kernels are used:

- An insertion kernel to set Bloom filter bits.
- A membership kernel to check all elements in parallel.

Key points:

- SipHash is fully ported to the GPU, including endian-safe 64-bit loads.
- Each thread maintains its own SipHash key and hash computations.
- The checking kernel uses atomicAdd to record false negatives.

The workflow is:

Generate strings → Insert on GPU → Membership check on GPU → Measure time

2.4 Timing Approach

CUDA events are used to measure GPU execution time (insert + check).

String generation time is excluded, as required.

CPU timing uses the standard gettimeofday() method.

3. GPU Architecture (GAIVI Cluster)

All experiments were run on a GAIVI node with:

- NVIDIA Titan X (Maxwell)
- 3072 CUDA cores
- 12 GB GDDR5
- CUDA 11.4 compiler toolchain

The Titan X architecture benefits this project because:

- SipHash is compute-intensive; massive parallel ALUs provide acceleration.
- Bloom filter accesses are mostly random but fit in global memory bandwidth.
- Each thread works independently with no inter-thread synchronization.

4. Experimental Results

All experiments were run with an error rate of 0.05 and block sizes of 256 or 512. All runs produced **zero false negatives**, confirming correctness of both CPU and GPU implementations.

4.1 CPU vs GPU Performance Table

Table 1. Execution Time and Speedup for Different Input Sizes (Block Size: 256)

+-----+-----+-----+-----+			
# Elements	CPU Time (ms)	GPU Time (ms)	Speedup
+-----+-----+-----+-----+			
10,000	19.402	0.127	153.2x
100,000	198.658	0.604	328.7x
1,000,000	1996.820	5.818	343.2x
+-----+-----+-----+-----+ All			

three runs correctly produced:

- CPU false negatives: 0
- GPU false negatives: 0

This confirms SipHash correctness on both host and device.

4.2 Block Size Comparison (100,000 Elements)

+-----+-----+-----+-----+			
Block Size	CPU Time (ms)	GPU Time (ms)	Speedup
+-----+-----+-----+-----+			
256	198.658	0.604	328.7x
512	198.620	0.624	318.3x +-----+--

+-----+-----+-----+-----+ Observation:

- Block size 256 performs slightly better than 512.
- The difference is small because the kernels are compute-bound (SipHash dominates cost).

5. Analysis and Discussion

5.1 Correctness

Across all datasets:

- CPU false negatives: 0
- GPU false negatives: 0

This is expected because the Bloom filter was filled and then immediately queried with the same dataset.

It also confirms:

- GPU SipHash implementation is correct.
- Thread-local keys and hashes are computed deterministically.
- No race conditions or memory hazards exist.

5.2 Performance Trends

The GPU consistently outperforms the CPU, with speedups ranging from 153x to 343x.

Reasons:

1. Each string is processed independently → massive parallelism.
2. SipHash is compute-heavy → GPU ALUs accelerate this significantly.
3. Memory accesses are simple (one byte per bit) → low contention.
4. No expensive synchronization inside kernels.

The gap widens as input size increases because the CPU grows linearly while the GPU remains nearly constant after launch overhead.

5.3 Block Size Effects

Block size 256 gives optimal performance.

Block size 512 results in slight register pressure (as seen in ptxas logs) but overall performance is very close.

Given SipHash's register usage (50 registers per thread), excessively large blocks reduce occupancy slightly.

5.4 CPU vs GPU comparison

The CPU baseline used for comparison is the instructor-provided **bloomfilter_CPU.cu**, which performs all SipHash computations sequentially on a single core. My GPU implementation parallelizes this work by assigning one string per thread and timing the insertion + membership checking using CUDA events. Both versions produced zero false

negatives, confirming correctness. Experimentally, the GPU achieves $153\times$ speedup at 10K elements, $328\times$ at 100K elements, and over $340\times$ at 1M elements. This performance gain occurs because the CPU performs hashing serially, while the GPU performs thousands of SipHash operations in parallel.

6. Conclusion

This project successfully implements a fully parallel Bloom filter using CUDA. Both CPU and GPU versions utilize SipHash-2-4 exactly as required. The GPU implementation provides significant speedup on the GAIVI Titan X GPU, reaching over **300x faster performance** while maintaining perfect correctness and zero false negatives.

The results confirm that Bloom filters are highly parallelizable and that cryptographic hash functions such as SipHash can be efficiently executed on GPUs.